Progress Report

Comparing LLM-Based Policy Generation and Reinforcement Learning for Embodied Control

Jarne Demoen

1 Week 1: Initial Pipeline and Baseline Evaluation

1.1 Goal of the Week

The goal of Week 1 was to create a minimal working experimental pipeline for the project. Instead of immediately starting with LLM-generated controllers, I first focused on a simple continuous-control environment. The main objective was to verify that I could create an environment, evaluate fixed controllers, train a reinforcement learning baseline, and save the results in a structured way.

1.2 Environment

The first environment used was `Pendulum-v1` from Gymnasium. I selected this environment because it has continuous observations and continuous actions. This makes it more relevant to the embodied-control setting of the project than a discrete task such as `CartPole`, while still being simple enough for debugging.

The observation contains three values:

- $\cos(\theta)$, the cosine of the pendulum angle;
- $\sin(\theta)$, the sine of the pendulum angle;
- $\dot{\theta}$, the angular velocity.

The action is one continuous torque value in the interval $[-2, 2]$. The goal is to apply torque so that the pendulum reaches and stays near the upright position. In this environment, rewards are usually negative, so values closer to zero indicate better performance.

1.3 Implemented Methods

Three methods were considered in the Week 1 report: a random controller, a manually defined controller, and a PPO baseline.

1.3.1 Random Controller

The random controller samples a torque uniformly from the valid action range at each time step. It does not use the observation and does not learn. It is included as a lower baseline to check whether other methods can do better than random actions.

1.3.2 Manual Controller

The manual controller is a fixed hand-written rule. It uses the pendulum angle and angular velocity to compute a torque. The angle is reconstructed from the observation using

$$\theta = \text{atan2}(\sin(\theta), \cos(\theta)).$$

The controller then applies a simple proportional-derivative style rule:

$$u = -2.0\theta - 0.5\dot{\theta}.$$

Here, u is the torque applied to the pendulum. The value of u is clipped to the valid torque range $[-2, 2]$. This controller is not trained; it is only a simple manually designed policy.

1.3.3 PPO Baseline

Proximal Policy Optimization, or PPO, is a policy-gradient reinforcement learning algorithm introduced by Schulman et al. [3]. It learns a parameterized policy $\pi_\theta(a \mid s)$, where s is the current state, a is the selected action, and θ represents the parameters of the neural network policy. During training, the agent interacts with the environment, collects trajectories, estimates which actions were better or worse than expected, and updates the policy.

The main idea behind PPO is to improve the policy without making updates that are too large. Large policy updates can make training unstable. PPO therefore compares the probability of an action under the new policy with the probability of the same action under the old policy. This probability ratio is defined as

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}.$$

PPO optimizes a clipped objective:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta)\hat{A}_t, \text{clip}\left(r_t(\theta), 1-\epsilon, 1+\epsilon\right)\hat{A}_t \right) \right].$$

Here, $\hat{A}_t$ is the estimated advantage and ϵ is the clipping parameter. The advantage indicates whether an action was better or worse than expected. The clipping term limits how much the new policy can deviate from the old policy during one update. This makes PPO relatively stable and easy to use compared to older policy-gradient methods.

The first reinforcement learning baseline was trained using Proximal Policy Optimization, implemented with Stable-Baselines3. PPO was included to check whether the pipeline can compare fixed controllers with a learned policy.

1.4 Evaluation Setup

The random and manual controllers were evaluated on `Pendulum-v1` for 100 episodes using seed 42. The PPO model was trained for 500,000 timesteps and also evaluated for 100 episodes. The reported metric is the mean cumulative reward over the evaluation episodes. Since rewards in `Pendulum-v1` are negative, higher values are better.

1.5 Results

Table 1 shows the Week 1 results.

Table 1: Week 1 results on `Pendulum-v1`. Higher reward is better.

Method	Episodes	Mean reward	Std. reward	Training
Random controller	100	−1239.407	293.894	No
Manual controller	100	−1180.920	234.175	No
PPO baseline	100	−182.502	107.747	Yes

The manual controller performed better than the random controller. The improvement in mean reward shows that the manually defined rule uses some useful information from the state. However, the improvement is limited. The PPO baseline obtained a mean reward of −182.502, which is substantially better than both fixed controllers. The current ranking is therefore

$$\text{PPO} > \text{Manual controller} > \text{Random controller}.$$

1.6 Comparison with Reported PPO Results

To put the obtained PPO result into context, I compared it with a publicly available Stable-Baselines3 PPO model for `Pendulum-v1` hosted on Hugging Face [4]. The reported result for that model is a mean reward of −230.423 with a standard deviation of 142.539, evaluated deterministically over 10 episodes.

In my pipeline, the PPO baseline obtained a mean reward of −182.502 with a standard deviation of 107.747. This is better than the reported Stable-Baselines3 result, since rewards in `Pendulum-v1` are negative and values closer to zero indicate better performance.

1.7 Discussion

The Week 1 results show that the experimental pipeline is functioning correctly. The random controller provides a useful lower baseline because it does not use the state and simply samples arbitrary actions. Its poor mean reward confirms that `Pendulum-v1` cannot be solved well by uncontrolled torque selection. The manual controller performs slightly better, indicating that even a simple rule based on angle and angular velocity can leverage meaningful information from the environment. However, the improvement over random control is relatively small. This suggests that manually designing a good controller is not trivial, even in a simple environment.

The PPO baseline performs substantially better than both fixed controllers. This is expected because PPO can improve its policy through interaction with the environment, while the random and manual controllers remain fixed. The large gap between PPO and the manual controller also illustrates why reinforcement learning is a strong baseline for embodied-control tasks. Instead

of relying on a manually chosen formula, PPO can adapt the policy parameters based on reward feedback. The comparison with the reported Stable-Baselines3 PPO result further supports this conclusion, where my PPO baseline performs better in this specific comparison. Nevertheless, the evaluation settings are not identical: the reported result uses 10 evaluation episodes, while my result uses 100 episodes. Because of this, the comparison should be treated as evidence that the implementation is reasonable and competitive, not as a definitive claim that my trained model is superior.

At the same time, the results show why comparing with LLM-based policy generation will be interesting. A manually written controller is easy to interpret, but its performance is limited. PPO gives much better performance, but the learned policy is less transparent because it is represented by neural network parameters. LLM-generated controllers may offer a different trade-off: they could produce readable control logic while still using more task knowledge than a very simple hand-written rule. Therefore, the next experiments should not only compare rewards but also consider factors such as interpretability, ease of modification, and the amount of human intervention needed.

Overall, Week 1 successfully establishes the basic structure needed for the project. The environment can be created, controllers evaluated, PPO trained, and results stored and compared. This creates a foundation for the next phase, where LLM-based policy generation can be added to the same evaluation pipeline.

1.8 Conclusion

In Week 1, I implemented and evaluated a first control pipeline for `Pendulum-v1`. The pipeline includes a random controller, a simple manually defined controller, and a PPO reinforcement learning baseline. The results show a clear performance ranking: PPO performs best, followed by the manual controller, while the random controller performs worst.

The main technical outcome is that the pipeline is working end-to-end. I can now evaluate fixed controllers, train a reinforcement learning agent, compute mean and standard deviation over multiple episodes, and compare the results in a structured way. This is an important first step because future LLM-generated controllers can be evaluated using the same setup.

The results also clarify the role of the different baselines. The random controller serves as a lower bound, the manual controller represents a simple interpretable policy, and PPO represents a strong learned reinforcement learning baseline.

2 Week 2: Reinforcement Learning Baselines and Reproducibility

2.1 Goal of the Week

The goal of Week 2 was to extend the initial reinforcement learning pipeline by implementing or configuring three reinforcement learning baselines: PPO, SAC, and DDPG. In Week 1, PPO was already implemented and evaluated on `Pendulum-v1`. In Week 2, the focus was therefore on adding SAC and DDPG, running initial experiments on the same simple continuous-control environment, defining reward thresholds, and verifying that experiments can be reproduced using fixed seeds.

2.2 Implemented Reinforcement Learning Baselines

Three reinforcement learning algorithms are now part of the baseline pipeline: PPO, SAC, and DDPG. These methods were selected because they are commonly used for continuous-control problems and can be applied directly to environments with continuous action spaces such as `Pendulum-v1`.

PPO was already used in Week 1 as the first learned reinforcement learning baseline. It is an on-policy algorithm that updates the policy using recently collected trajectories and controls the size of policy updates through a clipped objective. This makes it a stable and widely used baseline for reinforcement learning experiments.

Soft Actor-Critic, or SAC, was added as a second baseline. SAC is an off-policy actor-critic algorithm based on the maximum entropy reinforcement learning framework [1]. In addition to maximizing the expected return, SAC also maximizes the entropy of the policy. This encourages exploration because the policy is rewarded for remaining sufficiently stochastic during training. This can be useful in continuous-control tasks, where poor exploration may prevent the agent from discovering better actions.

Deep Deterministic Policy Gradient, or DDPG, was added as a third baseline. DDPG is an off-policy actor-critic algorithm for continuous action spaces [2]. It learns both a Q-function and a deterministic policy. The Q-function estimates the quality of state-action pairs, while the policy learns to output the continuous action that maximizes the Q-function. DDPG is relevant for this project because it is designed for continuous action spaces, but it can also be more sensitive to hyperparameters and exploration noise than newer algorithms such as SAC.

2.3 Experimental Setup

All Week 2 experiments were run on `Pendulum-v1`. This keeps the comparison consistent with Week 1 and makes it easier to verify whether the different reinforcement learning baselines behave as expected on a simple continuous-control task.

The PPO result is taken from the Week 1 experiment, where PPO was trained for 500,000 timesteps and evaluated for 100 episodes. SAC and DDPG were trained for 20,000 timesteps as initial experiments. The same evaluation principle was used: the reported metric is the mean cumulative reward, together with the standard deviation. Since rewards in `Pendulum-v1` are negative, values closer to zero indicate better performance.

To support reproducibility, fixed seeds were used in the experimental pipeline. This is important because reinforcement learning results can vary due to random initialization, environment randomness, exploration behavior, and stochastic training updates. Using fixed seeds makes it easier to debug the pipeline and to check whether repeated runs produce comparable results.

2.4 Reward Thresholds

To interpret the results more clearly, I defined simple reward thresholds for `Pendulum-v1`. These thresholds are not official success criteria, but they provide a practical way to classify the quality of the learned policies during the project.

- Mean reward below -1000: poor performance. The controller is close to random or does not solve the task meaningfully.

- Mean reward between -1000 and -500: limited performance. The controller shows some improvement but is still far from a good policy.
- Mean reward between -500 and -250: reasonable performance. The controller has learned useful behavior, but there is still room for improvement.
- Mean reward above -250: strong performance. The controller performs well on the task.

Using these thresholds, the random and manual controllers from Week 1 are classified as poor, while PPO, SAC, and DDPG all reach the strong-performance range.

2.5 Results

Table 2 shows the current reinforcement learning baseline results on `Pendulum-v1`.

Table 2: Week 2 reinforcement learning baseline results on `Pendulum-v1`. Higher reward is better.

Method	Total timesteps	Mean reward	Std. reward
PPO	500,000	-182.502	107.747
SAC	20,000	-149.033	73.988
DDPG	20,000	-153.602	76.448

The results show that all three reinforcement learning baselines perform much better than the random and manual controllers from Week 1. SAC obtained the best mean reward, with a score of -149.033. DDPG followed closely with a mean reward of -153.602. PPO obtained a mean reward of -182.502.

The current ranking among the reinforcement learning baselines is therefore

$$\text{SAC} > \text{DDPG} > \text{PPO}.$$

However, this ranking should be interpreted carefully because the training budgets are not identical. PPO was trained for 500,000 timesteps, while SAC and DDPG were trained for 20,000 timesteps. Therefore, the table is useful as an initial comparison of working baselines, but it is not yet a fully controlled benchmark. A more rigorous comparison should train all algorithms under the same budget and preferably repeat each experiment over multiple seeds.

2.6 Discussion

The Week 2 results show that the reinforcement learning baseline pipeline has been successfully extended beyond PPO. Both SAC and DDPG were configured and trained on the same environment, and both obtained strong results according to the defined reward thresholds. This is an important step because the project now has multiple learned baselines instead of relying on a single reinforcement learning method.

SAC achieved the best result in the current experiments. This is plausible because SAC is often strong in continuous-control tasks due to its off-policy learning and entropy-based exploration. The entropy term encourages the agent to explore different actions during training, which can help it discover better control behavior for the pendulum. In this experiment, SAC reached a mean reward of -149.033, which is the best result obtained so far.

DDPG also performed well, with a mean reward of -153.602. Its performance is close to SAC in this initial experiment. This shows that a deterministic actor-critic approach can also learn a useful policy for `Pendulum-v1`. However, DDPG is generally more sensitive to exploration noise and hyperparameter settings. For this reason, its stability should be tested further with repeated runs and different seeds.

PPO remains a useful baseline, even though it did not obtain the best mean reward in this comparison. PPO is stable and simple to use, and it was already shown in Week 1 to perform much better than the random and manual controllers. The current comparison mainly shows that SAC and DDPG can also be successfully integrated into the pipeline and can reach competitive or better performance on the same task.

The main limitation of the Week 2 comparison is that the training budgets are different. PPO was trained for many more timesteps than SAC and DDPG, so the results should not be interpreted as a definitive algorithm ranking. Instead, Week 2 should be seen as an implementation and verification phase. The key outcome is that PPO, SAC, and DDPG all run correctly, produce meaningful results, and can be evaluated using the same pipeline.

2.7 Conclusion

In Week 2, I extended the experimental pipeline by adding SAC and DDPG alongside the PPO baseline from Week 1. All three reinforcement learning algorithms were evaluated on `Pendulum-v1`. The results show that SAC, DDPG, and PPO all achieve strong performance according to the defined reward thresholds.

The best result in the current experiments was obtained by SAC, followed closely by DDPG. PPO also remained clearly stronger than the fixed random and manual controllers from Week 1. These results confirm that the reinforcement learning part of the project is functioning correctly and that the pipeline can support multiple baseline algorithms.

The main next step is to make the comparison more systematic. Future experiments should use equal training budgets for PPO, SAC, and DDPG, evaluate each method over multiple seeds, and then compare the algorithms more fairly. After that, LLM-generated controllers can be added to the same evaluation setup and compared against these reinforcement learning baselines.

References

[1] Haarnoja, T., Zhou, A., Abbeel, P., and Levine, S. Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. arXiv preprint arXiv:1801.01290, 2018.

[2] OpenAI Spinning Up. Deep Deterministic Policy Gradient. Available at: https://spinningup.openai.com/en/latest/algorithms/ddpg.html.

[3] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal Policy Optimization Algorithms. arXiv preprint arXiv:1707.06347, 2017.

[4] Stable-Baselines3. PPO Agent Playing `Pendulum-v1`: Results JSON. Hugging Face, 2024. Available at: https://huggingface.co/sb3/ppo-Pendulum-v1/blob/main/results.json.

[5] Demoen, J. Comparing LLM-Based Policy Generation and Reinforcement Learning for Embodied Control. Estudo Dirigido Project Proposal, 2026.